

Week 19 – Mock Contest 2

■ Contest Setup (1.5 hrs, 3 mixed problems)

- Expect a twist: parsing or 2D traversal.
- Read all tasks first; identify quick wins and partial points.
- Budget time; leave 10 minutes at end for final checks.

Problem A – Word Compression (J2)

Compress consecutive duplicate characters (e.g., aaabb → a3b2).

Solution:

```
s = input().strip()
if not s:
    print("")
else:
    out = []
    cur = s[0]; cnt = 1
    for ch in s[1:]:
        if ch == cur:
            cnt += 1
        else:
            out.append(cur + str(cnt))
            cur, cnt = ch, 1
    out.append(cur + str(cnt))
    print(''.join(out))
```

Problem B – Meeting Rooms (Greedy)

Given intervals, find min number of rooms required.

Solution:

```
intervals = [(0,30),(5,10),(15,20)]
events = []
for s,e in intervals:
    events.append((s, 1))
    events.append((e, -1))
events.sort()
cur = ans = 0
for _, d in events:
    cur += d
    ans = max(ans, cur)
print(ans)
```

Problem C – Shortest Path in Grid (BFS)

Find shortest steps from (0,0) to (R-1,C-1) avoiding '#'.

Solution:

```
from collections import deque
R, C = map(int, input().split())
grid = [list(input().strip()) for _ in range(R)]
dist = [[-1]*C for _ in range(R)]
q = deque([(0,0)])
dist[0][0] = 0
dirs = [(-1,0),(1,0),(0,-1),(0,1)]
while q:
    r,c = q.popleft()
    for dr,dc in dirs:
        nr, nc = r+dr, c+dc
        if 0<=nr<R and 0<=nc<C and grid[nr][nc] != '#' and dist[nr][nc] == -1:
            dist[nr][nc] = dist[r][c] + 1
            q.append((nr,nc))
print(dist[R-1][C-1])
```

■ Debrief & Strategy

- Partial scoring: implement simpler version first (e.g., smaller limits).
- Validate parsing carefully; trim spaces and handle empty lines.
- Prefer BFS for shortest path in unweighted grids.